

Sistema de Gestión para Heladería Cafetería con Módulo Integrado de Monitoreo de Consumo de Agua y Energía

Universidad: [PENDIENTE: universidad]

Facultad: [PENDIENTE: facultad]

Carrera: [PENDIENTE: carrera]

Asignatura: Programación Web II

Integrantes: [PENDIENTE: integrantes]

Docente: [PENDIENTE: docente]

Gestión y fecha: [PENDIENTE: gestión y fecha]

Hoja de control documental

Versión	Fecha	Descripción del cambio	Responsable
1.0	2026-09-30	Elaboración inicial basada en el repositorio	[PENDIENTE]

Índice general

Capítulo I. Introducción; Capítulo II. Ingeniería de requisitos; Capítulo III. Modelado UML; Capítulo IV. Base de datos; Capítulo V. Arquitectura; Capítulo VI. Desarrollo; Capítulo VII. Docker; Capítulo VIII. Investigación; Capítulo IX. Pruebas; Capítulo X. Instalación; Capítulo XI. Despliegue; Capítulo XII. Conclusiones; referencias y anexos.

Índice de figuras

N.º	Figura
1	Casos de uso del POS
2	Casos de uso del módulo de monitoreo
3	Modelo entidad relación del POS
4	Modelo entidad relación del monitoreo

CAPÍTULO I. INTRODUCCIÓN

Antecedentes

La operación de una heladería/cafetería integra ventas, caja, pedidos, inventario y atención al cliente. El proyecto centraliza estos procesos y añade una aplicación autónoma para registrar, clasificar y alertar sobre el consumo de agua y energía.

Planteamiento del problema

Se requiere preservar la trazabilidad de ventas, pagos, stock y turnos, a la vez que se registran consumos operativos. La integración debe tolerar indisponibilidad temporal sin compartir tablas entre aplicaciones.

Justificación

La propuesta aplica arquitectura por capas, API REST, transacciones, JWT, RBAC, RLS y contenedores. Contribuye al uso responsable de recursos y permite aplicar conocimientos de programación web y bases de datos.

Objetivos

Objetivo general: desarrollar un sistema de gestión para heladería/cafetería integrado por API con un módulo de monitoreo de agua y energía. Objetivos específicos: registrar operación comercial, trazabilidad e inventario; reportar consumo; clasificar y alertar; y aplicar seguridad e infraestructura reproducible.

Alcance y ODS

Incluye POS, Monitoreo, React, Express, Sequelize, PostgreSQL/Supabase, contratos, Docker y manifiestos Kubernetes. Se alinea con ODS 6, ODS 7 y ODS 12. La URL pública y la evidencia de despliegue permanecen pendientes.

CAPÍTULO II. INGENIERÍA DE REQUERIMIENTOS

Requerimientos funcionales

Código	Descripción	Módulo
RF01	Registrar venta normal, multiproducto y pagos divididos	Ventas
RF02	Rechazar venta sin stock; anular y devolver con reposición	Ventas/inventario
RF03	Abrir/cerrar turno, calcular diferencia y consumo	Caja
RF04	Gestionar mesas y pedidos	Pedidos
RF05	Gestionar inventario, recetas, clientes y promociones	POS
RF06	Enviar consumos y recibir alertas de forma idempotente	Integración
RF07	Registrar, clasificar y alertar consumos	Monitoreo
RF08	Gestionar metas, tarifas, notificaciones y recomendaciones	Monitoreo

Requerimientos no funcionales

Código	Categoría	Criterio
RNF01	Seguridad	JWT, RBAC, RLS, validación y rate limiting.
RNF02	Integridad	Transacciones y funciones PostgreSQL para operaciones críticas.
RNF03	Disponibilidad	Colas, reintentos, backoff y health checks.
RNF04	Trazabilidad	Auditoría y movimientos.
RNF05	Portabilidad	Docker y Kubernetes.
RNF06	Mantenibilidad	Capas, contratos y módulos separados.

Reglas de negocio y restricciones

Un cajero mantiene un único turno abierto. Las ventas anuladas conservan historial. Las devoluciones no pueden superar lo vendido. El stock, los insumos y los puntos dejan

trazabilidad. Los consumos y alertas se procesan idempotentemente. Los umbrales no se solapan, las metas se limitan a 0-100 %, y POS/Monitoreo no tienen claves foráneas cruzadas.

CAPÍTULO III. MODELADO UML

La Figura 1 representa los casos de uso del POS y la Figura 2 los del monitoreo. Los actores identificados incluyen cajero, mesero, encargado de inventario, administrador, supervisor, responsable de recursos, sistema POS y sistema de monitoreo.

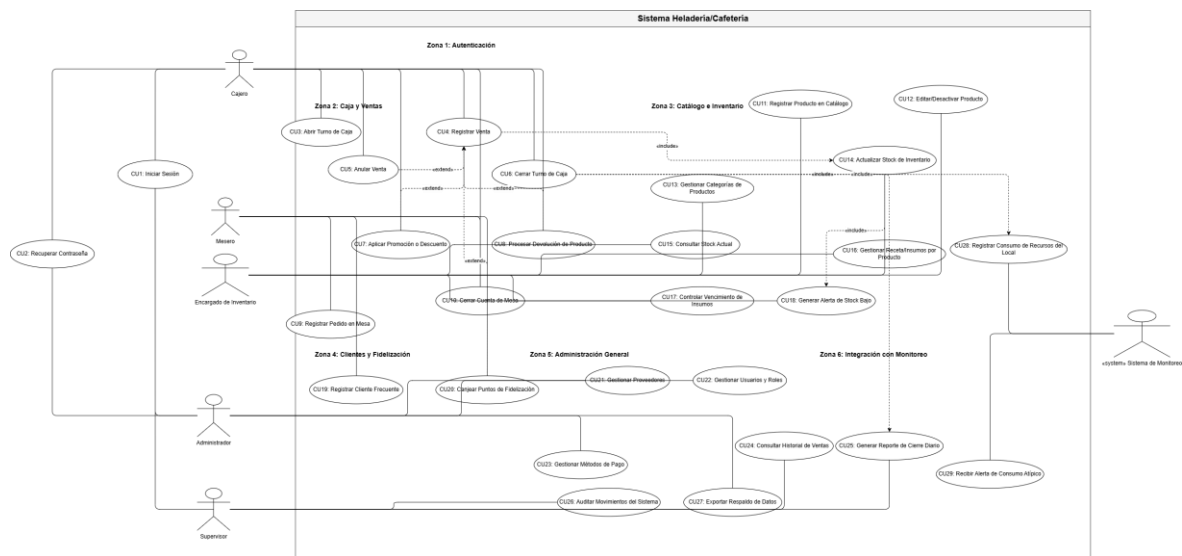


Figura 1. Diagrama de casos de uso del POS. Fuente: repositorio del proyecto.

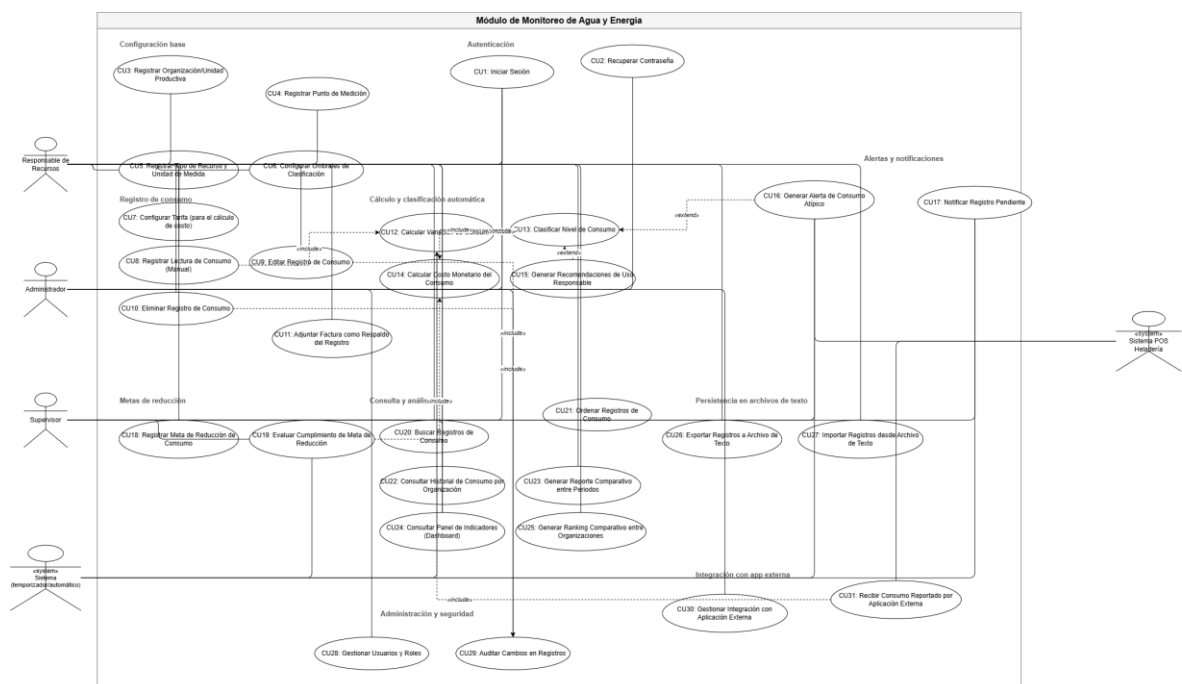


Figura 2. Diagrama de casos de uso del módulo de monitoreo. Fuente: repositorio del proyecto.

Descripción de casos de uso críticos

Caso	Actor	Precondición	Flujo	Alternativo	Postcondición
Registrar venta	Cajero	JWT, permiso, turno	Valida stock/pagos y registra	Stock o pago inválido	Venta y auditoría
Cerrar turno	Cajero	Turno abierto	Calcula efectivo y consumo	Monto inválido	Turno cerrado
Anular venta	Supervisor	Venta activa	Restituye existencias	Ya anulada	Historial conservado
Procesar devolución	Usuario autorizado	Venta válida	Valida cantidades y restituye	Supera vendido	Devolución trazable
Recibir consumo	POS	Credencial integración	Valida y encola	Duplicado	Una recepción
Clasificar consumo	Worker	Registro pendiente	Compara umbral	Sin umbral	Nivel asignado
Generar alerta	Worker	Exceso	Crea y encola alerta	Sin destinatario	Entrega pendiente

Diagrama de clases

En POS, Usuario, Rol y Permiso sustentan el acceso; Venta, DetalleVenta, Pago y TurnoCaja modelan operación; Producto, Insumo, RecetaInsumo y MovimientoInventario modelan existencias. En Monitoreo, Organizacion, PuntoMedicion, TipoRecurso, RegistroConsumo, UmbralClasificacion y Alerta constituyen el núcleo. Las clases de colas y recepción aíslan la integración REST.

CAPÍTULO IV. BASE DE DATOS

Las Figuras 3 y 4 describen bases independientes sin claves foráneas cruzadas. Se priorizan UUID y NUMERIC para claves, dinero y cantidades.

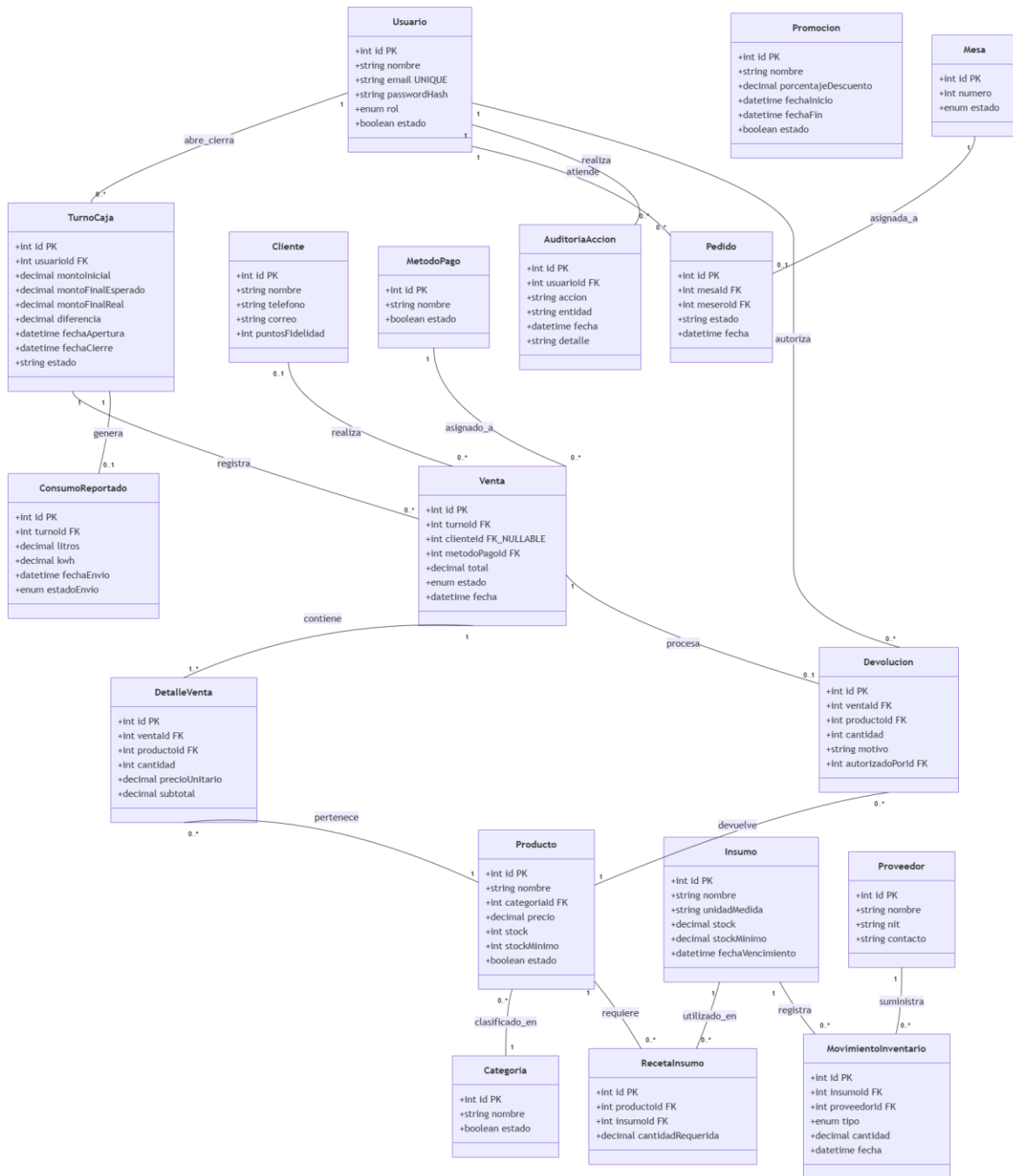


Figura 3. Modelo entidad relación del POS. Fuente: repositorio del proyecto.

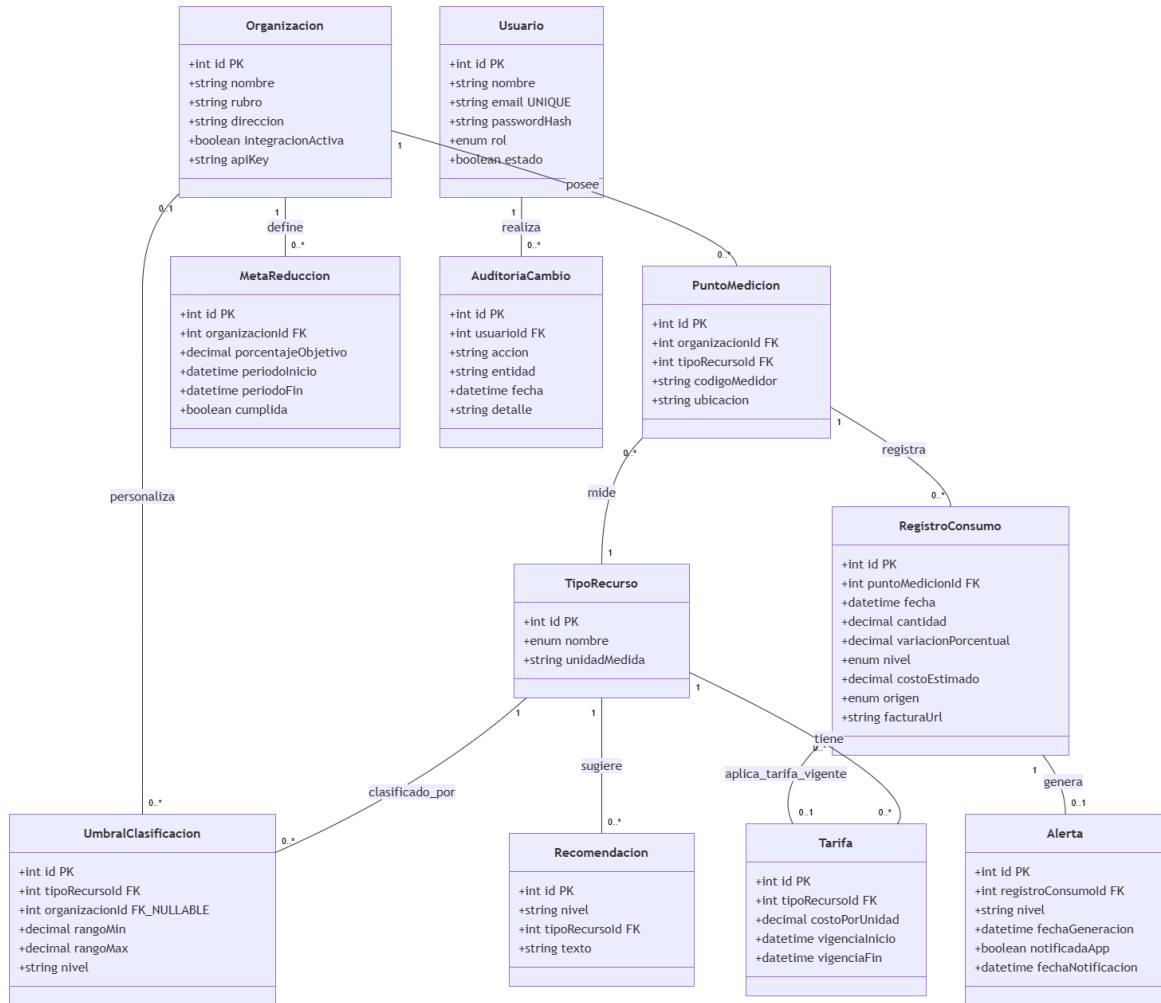


Figura 4. Modelo entidad relación del monitoreo. Fuente: repositorio del proyecto.

Diccionario de datos

POS

alerta_pos - AlertaPos.js

Columna	Tipo	Restricción
id_alerta	UUID	PK
turno_id	UUID	FK [ver asociación Sequelize]
tipo	STRING	NN
nivel	STRING	NN
mensaje	TEXT	NN
estado	STRING	—
creado_en	DATE	—
atendido_en	DATE	—
atendido_por	UUID	—

auditoria_accion - AuditoriaAccion.js

Columna	Tipo	Restricción
id_auditoria	UUID	PK
usuario_id	UUID	FK [ver asociación Sequelize]
accion	STRING	NN
entidad	STRING	—
entidad_id	UUID	FK [ver asociación Sequelize]
resultado	STRING	—
direccion_ip	STRING	—
user_agent	STRING	—
fecha	DATE	—
detalle	TEXT	—

categoria - Categoria.js

Columna	Tipo	Restricción
id_categoria	UUID	PK
nombre	STRING	NN
estado	STRING	—

cliente - Cliente.js

Columna	Tipo	Restricción
id_cliente	UUID	PK
nombre	STRING	NN
telefono	STRING	—
correo	STRING	—
puntos_fidelidad	INTEGER	—
estado	STRING	—
creado_en	DATE	Generado por Sequelize
actualizado_en	DATE	Generado por Sequelize

cola_integracion - ColaIntegracion.js

Columna	Tipo	Restricción
id_cola	UUID	PK
consumo_id	UUID	NN; FK [ver asociación Sequelize]
operacion	STRING	NN
idempotency_key	STRING	NN
intentos	INTEGER	—
estado	STRING	—
respuesta	TEXT	—
error	TEXT	—
proximo_intento	DATE	—
ultimo_intento	DATE	—
creado_en	DATE	—

configuracion_pos - ConfiguracionPos.js

Columna	Tipo	Restricción
id_configuracion	UUID	PK
clave	STRING	NN
valor	STRING	NN
descripcion	TEXT	—
actualizado_en	DATE	—

consumo_reportado - ConsumoReportado.js

Columna	Tipo	Restricción
id_consumo	UUID	PK
turno_id	UUID	NN; FK [ver asociación Sequelize]
tipo_recurso	STRING	NN
cantidad	DECIMAL	NN
unidad_medida	STRING	NN
fecha_consumo	DATE	—
estado	STRING	—
fecha_creacion	DATE	—

detalle_pedido - DetallePedido.js

Columna	Tipo	Restricción
id_detalle_pedido	UUID	PK
pedido_id	UUID	NN; FK [ver asociación Sequelize]
producto_id	UUID	NN; FK [ver asociación Sequelize]
cantidad	DECIMAL	NN
precio_unitario	DECIMAL	NN
observacion	TEXT	—
subtotal	DECIMAL	NN

detalle_venta - DetalleVenta.js

Columna	Tipo	Restricción
id_detalle_venta	UUID	PK
venta_id	UUID	NN; FK [ver asociación Sequelize]
producto_id	UUID	NN; FK [ver asociación Sequelize]
cantidad	DECIMAL	NN
precio_unitario	DECIMAL	NN
descuento	DECIMAL	—
subtotal	DECIMAL	NN

devolucion - Devolucion.js

Columna	Tipo	Restricción
id_devolucion	UUID	PK
venta_id	UUID	NN; FK [ver asociación Sequelize]
producto_id	UUID	NN; FK [ver asociación Sequelize]
autorizado_por_id	UUID	FK [ver asociación Sequelize]
cantidad	DECIMAL	NN
monto	DECIMAL	NN
motivo	TEXT	—
estado	STRING	—
fecha	DATE	—

entrega_alerta - EntregaAlerta.js

Columna	Tipo	Restricción
id_entrega	UUID	PK
alerta_externa_id	UUID	NN; FK [ver asociación Sequelize]
nivel	STRING	NN
tipo	STRING	NN

Columna	Tipo	Restricción
mensaje	TEXT	NN
estado	STRING	—
intentos	INTEGER	—
fecha_recepcion	DATE	—
fecha_procesamiento	DATE	—

equipo_consumo - EquipoConsumo.js

Columna	Tipo	Restricción
id_equipo	UUID	PK
nombre	STRING	NN
tipo_recurso	STRING	NN
consumo_por_hora	DECIMAL	NN
unidad_medida	STRING	NN
activo	BOOLEAN	—
creado_en	DATE	Generado por Sequelize
actualizado_en	DATE	Generado por Sequelize

equipo_turno - EquipoTurno.js

Columna	Tipo	Restricción
id_equipo_turno	UUID	PK
equipo_id	UUID	NN; FK [ver asociación Sequelize]
turno_id	UUID	NN; FK [ver asociación Sequelize]
hora_inicio	DATE	NN
hora_fin	DATE	—
estado	STRING	—
creado_en	DATE	Generado por Sequelize
actualizado_en	DATE	Generado por Sequelize

insumo - Insumo.js

Columna	Tipo	Restricción
id_insumo	UUID	PK
nombre	STRING	NN
unidad_medida	STRING	—
stock	DECIMAL	—
stock_minimo	DECIMAL	—
fecha_vencimiento	DATE	—
estado	STRING	—

mesa - Mesa.js

Columna	Tipo	Restricción
id_mesa	UUID	PK
numero	INTEGER	NN
estado	STRING	—

metodo_pago - MetodoPago.js

Columna	Tipo	Restricción
id_metodo_pago	UUID	PK
nombre	STRING	NN
estado	STRING	—

movimiento_inventario - MovimientoInventario.js

Columna	Tipo	Restricción
id_movimiento	UUID	PK
insumo_id	UUID	FK [ver asociación Sequelize]
proveedor_id	UUID	FK [ver asociación Sequelize]
usuario_id	UUID	FK [ver asociación Sequelize]
producto_id	UUID	FK [ver asociación Sequelize]
venta_id	UUID	FK [ver asociación Sequelize]
devolucion_id	UUID	FK [ver asociación Sequelize]
tipo	STRING	NN
cantidad	DECIMAL	NN
motivo	STRING	—
fecha	DATE	—

movimiento_puntos - MovimientoPuntos.js

Columna	Tipo	Restricción
id_movimiento	UUID	PK
cliente_id	UUID	NN; FK [ver asociación Sequelize]
venta_id	UUID	FK [ver asociación Sequelize]
puntos	INTEGER	NN
tipo	STRING	NN
motivo	STRING	—
fecha	DATE	—

pago - Pago.js

Columna	Tipo	Restricción
id_pago	UUID	PK
venta_id	UUID	NN; FK [ver asociación Sequelize]
metodo_pago_id	UUID	NN; FK [ver asociación Sequelize]
monto	DECIMAL	NN
referencia	STRING	—
estado	STRING	—
fecha	DATE	—

pedido - Pedido.js

Columna	Tipo	Restricción
id_pedido	UUID	PK
mesa_id	UUID	FK [ver asociación Sequelize]
mesero_id	UUID	FK [ver asociación Sequelize]
estado	STRING	—
fecha	DATE	—
fecha_cierre	DATE	—

permiso - Permiso.js

Columna	Tipo	Restricción
id_permiso	UUID	PK
nombre	STRING	NN
descripcion	STRING	—
modulo	STRING	—

producto - Producto.js

Columna	Tipo	Restricción
id_producto	UUID	PK
categoria_id	UUID	FK [ver asociación Sequelize]
nombre	STRING	NN
precio	DECIMAL	NN
imagen_url	TEXT	—
stock	DECIMAL	—
stock_minimo	DECIMAL	—
estado	STRING	—
creado_en	DATE	Generado por Sequelize
actualizado_en	DATE	Generado por Sequelize

promocion - Promocion.js

Columna	Tipo	Restricción
id_promocion	UUID	PK
nombre	STRING	NN
porcentaje_descuento	DECIMAL	NN
fecha_inicio	DATE	—
fecha_fin	DATE	—
estado	STRING	—

promocion_producto - PromocionProducto.js

Columna	Tipo	Restricción
id_promocion_producto	UUID	PK
promocion_id	UUID	NN; FK [ver asociación Sequelize]
producto_id	UUID	NN; FK [ver asociación Sequelize]

proveedor - Proveedor.js

Columna	Tipo	Restricción
id_proveedor	UUID	PK
nombre	STRING	NN
nit	STRING	—
contacto	STRING	—
telefono	STRING	—
correo	STRING	—
estado	STRING	—

receta_ingreso - RecetaIngreso.js

Columna	Tipo	Restricción
id_receta	UUID	PK
producto_id	UUID	NN; FK [ver asociación Sequelize]
ingreso_id	UUID	NN; FK [ver asociación Sequelize]
cantidad_requerida	DECIMAL	NN

rol - Rol.js

Columna	Tipo	Restricción
id_rol	UUID	PK
nombre	STRING	NN
descripcion	STRING	—
estado	STRING	—

rol_permiso - RolPermiso.js

Columna	Tipo	Restricción
id_rol_permiso	UUID	PK
rol_id	UUID	NN; FK [ver asociación Sequelize]
permiso_id	UUID	NN; FK [ver asociación Sequelize]

turno_caja - TurnoCaja.js

Columna	Tipo	Restricción
id_turno	UUID	PK
usuario_id	UUID	NN; FK [ver asociación Sequelize]
monto_inicial	DECIMAL	NN
monto_final_esperado	DECIMAL	—
monto_final_real	DECIMAL	—
diferencia	DECIMAL	—
fecha_apertura	DATE	—
fecha_cierre	DATE	—
estado	STRING	—

users - Usuario.js

Columna	Tipo	Restricción
nombre	STRING	NN
email	STRING	NN
rol_id	UUID	FK [ver asociación Sequelize]
estado	STRING	—
creado_en	DATE	Generado por Sequelize
actualizado_en	DATE	Generado por Sequelize

venta - Venta.js

Columna	Tipo	Restricción
id_venta	UUID	PK
turno_id	UUID	NN; FK [ver asociación Sequelize]
cliente_id	UUID	FK [ver asociación Sequelize]
pedido_id	UUID	FK [ver asociación Sequelize]
subtotal	DECIMAL	—
descuento	DECIMAL	—
total	DECIMAL	NN
estado	STRING	—
motivo_anulacion	TEXT	—
fecha	DATE	—
idempotency_key	STRING	—

Monitoreo

organizacion - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
nombre	TEXT NOT	NN
nit	TEXT UNIQUE	—
estado	TEXT NOT	NN
creado_en	TIMESTAMPTZ NOT	NN

usuario - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
email	TEXT NOT	NN
nombre	TEXT	—
estado	TEXT NOT	NN
creado_en	TIMESTAMPTZ NOT	NN

rol - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
nombre	TEXT NOT	NN
estado	TEXT NOT	NN

permiso - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
nombre	TEXT NOT	NN

rol_permiso - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
rol_id	UUID NOT	NN; FK
permiso_id	UUID NOT	NN; FK

usuario_organizacion - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
usuario_id	UUID NOT	NN; FK
organizacion_id	UUID NOT	NN; FK
rol_id	UUID NOT	NN; FK
estado	TEXT NOT	NN
creado_en	TIMESTAMPTZ NOT	NN

integracion - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
organizacion_id	UUID NOT	NN; FK
nombre	TEXT NOT	NN
api_key_hash	TEXT NOT	NN
estado	TEXT NOT	NN
ultimo_uso_en	TIMESTAMP TZ	—
creado_en	TIMESTAMP TZ NOT	NN

tipo_recurso - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
codigo	TEXT NOT	NN
nombre	TEXT NOT	NN
unidad_base	TEXT NOT	NN

punto_medicion - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
organizacion_id	UUID NOT	NN; FK
tipo_recurso_id	UUID NOT	NN; FK
codigo_medidor	TEXT NOT	NN
nombre	TEXT NOT	NN
estado	TEXT NOT	NN
creado_en	TIMESTAMP TZ NOT	NN

recepcion_consumo_pos - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
consumo_externo_id	UUID NOT	NN
idempotency_key	TEXT NOT	NN
organizacion_id	UUID NOT	NN; FK
punto_medicion_id	UUID REFERENCES	FK
tipo_recurso	TEXT NOT	NN
cantidad	NUMERIC(14,3) NOT	NN
unidad_medida	TEXT NOT	NN
fecha_consumo	TIMESTAMP TZ NOT	NN
origen	TEXT NOT	NN
estado	TEXT NOT	NN
recepcionado_en	TIMESTAMP TZ NOT	NN

cola_procesamiento - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
recepcion_id	UUID NOT	NN; FK
estado	TEXT NOT	NN
intentos	INT NOT	NN
proximo_intento	TIMESTAMP TZ	—
ultimo_error	TEXT	—
creado_en	TIMESTAMP TZ NOT	NN

registro_consumo - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
repcion_id	UUID NOT	NN; FK
organizacion_id	UUID NOT	NN; FK
punto_medicion_id	UUID REFERENCES	FK
tipo_recurso_id	UUID NOT	NN; FK
tipo_recurso	TEXT NOT	NN
cantidad	NUMERIC(14,3) NOT	NN
unidad_medida	TEXT NOT	NN
fecha_consumo	TIMESTAMPTZ NOT	NN
clasificacion	TEXT NOT	NN
origen	TEXT NOT	NN
creado_en	TIMESTAMPTZ NOT	NN

umbral_clasificacion - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
organizacion_id	UUID NOT	NN; FK
tipo_recurso_id	UUID NOT	NN; FK
nombre	TEXT NOT	NN
nivel	TEXT NOT	NN
limite_inferior	NUMERIC(14,3) NOT	NN
limite_superior	NUMERIC(14,3) NOT	NN
estado	TEXT NOT	NN
creado_en	TIMESTAMPTZ NOT	NN

alerta - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
organizacion_id	UUID NOT	NN; FK
registro_consumo_id	UUID REFERENCES	FK
umbral_id	UUID REFERENCES	FK
nivel	TEXT NOT	NN
tipo_recurso	TEXT NOT	NN
mensaje	TEXT NOT	NN
fecha_generacion	TIMESTAMPTZ NOT	NN
estado	TEXT NOT	NN
creado_en	TIMESTAMPTZ NOT	NN

notificacion - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
alerta_id	UUID NOT	NN; FK
usuario_id	UUID REFERENCES	FK
canal	TEXT NOT	NN
estado	TEXT NOT	NN
creada_en	TIMESTAMPTZ NOT	NN
vista_en	TIMESTAMPTZ	—

entrega_alerta - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
alerta_id	UUID NOT	NN; FK
estado	TEXT NOT	NN
intentos	INT NOT	NN
proximo_intento	TIMESTAMPTZ	—
ultimo_error	TEXT	—
creada_en	TIMESTAMPTZ NOT	NN

meta_reduccion - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
organizacion_id	UUID NOT	NN; FK
tipo_recurso_id	UUID NOT	NN; FK
nombre	TEXT NOT	NN
porcentaje_reduccion	NUMERIC(5,2) NOT	NN
fecha_inicio	DATE NOT	NN
fecha_fin	DATE NOT	NN
estado	TEXT NOT	NN
creado_en	TIMESTAMPTZ NOT	NN

tarifa - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
organizacion_id	UUID REFERENCES	FK
tipo_recurso_id	UUID NOT	NN; FK
nombre	TEXT NOT	NN
monto	NUMERIC(14,4) NOT	NN
unidad	TEXT NOT	NN
fecha_inicio	DATE NOT	NN
fecha_fin	DATE NOT	NN
creado_en	TIMESTAMPTZ NOT	NN

recomendacion - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
organizacion_id	UUID NOT	NN; FK
titulo	TEXT NOT	NN
descripcion	TEXT NOT	NN
prioridad	TEXT NOT	NN
estado	TEXT NOT	NN
creada_en	TIMESTAMPTZ NOT	NN

auditoria_cambio - 001_v1_0_monitoreo_ddl.sql

Columna	Tipo	Restricción
id	UUID PRIMARY	PK
entidad	TEXT NOT	NN
entidad_id	UUID	—
accion	TEXT NOT	NN
usuario_id	UUID	—
req_id	TEXT	—

Columna	Tipo	Restricción
detalle	JSONB	—
creado_en	TIMESTAMPTZ NOT	NN

CAPÍTULO V. ARQUITECTURA DEL SISTEMA

La arquitectura prevista es routes → controllers → services → repositories → models. La documentación registra handlers inline en rutas del POS, por lo que se reporta como diferencia de implementación. Docker Compose coordina cuatro servicios y Supabase provee la base externa. Kubernetes contiene recursos de despliegue; AWS está previsto sin una arquitectura IaC verificable.

Flujo de integración

POS cierra turno, calcula y guarda ConsumoReportado; ColaIntegracion y un worker envían el contrato consumption.v1 a Monitoreo. Allí se registra la recepción idempotente, se procesa, clasifica y genera alerta. La alerta se entrega al POS mediante alerts.v1, evitando duplicados.

CAPÍTULO VI. DESARROLLO DEL SISTEMA

Capa	Tecnología	Evidencia
Frontend	React 18/19 y Vite	pos/frontend, monitoreo/frontend
Backend	Node.js y Express 5	app.js
ORM	Sequelize 6	src/models
Datos	PostgreSQL/Supabase	migraciones
Seguridad	JWT, RBAC, RLS, zod, helmet, cors	middlewares/shared docs
Infraestructura	Docker, Kubernetes, AWS previsto	infrastructure

La plantilla académica que menciona Laravel se traduce al stack real Node.js/Express/React/PostgreSQL. No se atribuye PHP, Blade, Eloquent o MySQL al repositorio. Los modelos Sequelize, rutas, middleware y páginas React cumplen los roles equivalentes.

CAPÍTULO VII. IMPLEMENTACIÓN CON DOCKER

Se verificaron cuatro Dockerfiles: POS backend y frontend, Monitoreo backend y frontend. docker-compose.yml expone 3000, 4000, 5173 y 5174; incluye health checks y límites de recursos. Se añadieron .dockerignore en los cuatro contextos para evitar copiar dependencias, secretos y artefactos.

[INSERTAR CAPTURA: docker compose build. Ruta: docs/proyecto-final/evidencias/07-docker-build.png.]

[INSERTAR CAPTURA: docker compose up -d. Ruta: docs/proyecto-final/evidencias/07-docker-up.png.]

[INSERTAR CAPTURA: docker compose ps. Ruta: docs/proyecto-final/evidencias/07-docker-ps.png.]

[INSERTAR CAPTURA: docker compose logs. Ruta: docs/proyecto-final/evidencias/07-docker-logs.png.]

[INSERTAR CAPTURA: sistema en Docker. Ruta: docs/proyecto-final/evidencias/07-sistema-docker.png.]

CAPÍTULO VIII. INVESTIGACIÓN EN FORMATO IMRyD

Introducción y metodología

El hosting publica aplicaciones en red y Docker las distribuye como imágenes reproducibles. Se compararon Render, Railway y Koyeb a través de sus sitios oficiales por compatibilidad Docker, gestión desde repositorio, HTTPS, servicios, secretos y limitaciones de plan.

Resultados y discusión

Plataforma	Docker	Conclusión
Render	Construye desde Dockerfile o usa imagen	Apta para servicios individuales.
Railway	Usa Dockerfile; traduce cada servicio Compose	Apta si se crean cuatro servicios.
Koyeb	Admite contenedores o repositorios Git	Alternativa contenerizada.

[PENDIENTE: plataforma elegida, URL pública y evidencia.] No se afirma un despliegue no verificado.

CAPÍTULO IX. PRUEBAS

Existe un runbook integrado de 17 pasos para turnos, ventas, consumos, idempotencia, alertas, anulación y auditoría. Se identificaron pruebas Jest/Vitest en POS y unitarias/integración en Monitoreo. Los resultados reales requieren ejecución.

ID	Caso	Esperado	Resultado	Estado
CP01	Venta y pago dividido	Venta y total válidos	[PENDIENTE]	Pendiente

ID	Caso	Esperado	Resultado	Estado
CP02	Stock insuficiente	Rechazo sin cambios	[PENDIENTE]	Pendiente
CP03	Anulación/devolución	Stock restaurado	[PENDIENTE]	Pendiente
CP04	Cierre de turno	Diferencia y consumos	[PENDIENTE]	Pendiente
CP05	Idempotencia	Sin duplicados	[PENDIENTE]	Pendiente
CP06	Seguridad	Rechazo JWT/payload/permiso	[PENDIENTE]	Pendiente
CP07	Resiliencia	Reintento y recuperación	[PENDIENTE]	Pendiente

[INSERTAR CAPTURA: ejecución de pruebas npm test en cada aplicación. Ruta: docs/proyecto-final/evidencias/09-pruebas.png.]

CAPÍTULO X. MANUAL DE INSTALACIÓN

Instalar Docker Desktop. Copiar .env.example a .env y completar secretos sin publicarlos. Desde la raíz ejecutar docker compose build y docker compose up -d. Verificar con docker compose ps y docker compose logs. Abrir http://localhost:5173 (POS), http://localhost:5174 (Monitoreo), y usar puertos 3000/4000 para backends. Supabase debe tener las migraciones y credenciales disponibles.

CAPÍTULO XI. MANUAL DE DESPLIEGUE

Seleccionar una plataforma, crear cuatro servicios a partir de sus Dockerfiles, cargar secretos en su gestor de variables, configurar URLs internas, CORS y health checks, publicar frontends y ejecutar la prueba integrada. [PENDIENTE: plataforma, URL y capturas de despliegue.]

CAPÍTULO XII. CONCLUSIONES Y RECOMENDACIONES

El repositorio implementa dos aplicaciones separadas que integran operación comercial y monitoreo por API y colas. La separación de datos, idempotencia y trazabilidad son decisiones centrales. Se recomienda ejecutar la matriz de pruebas, unificar el puerto 4000/4001, formalizar la migración canónica, completar IaC/AWS si procede, añadir CI/CD y publicar evidencia de un despliegue real.

Inconsistencias detectadas

Hallazgo	Tratamiento en este informe
Puerto de Monitoreo	README indica 4001; env, Compose y K8s indican 4000. Se usa 4000.
Capa de controladores	La arquitectura la prevé; el POS documenta handlers inline.
Conteo de pruebas	Fuentes difieren; no se afirma conteo único sin ejecución.
Migraciones POS	Se adopta database/pos/migrations como canónica conforme a documentación.

Referencias bibliográficas

Docker. (s. f.). Docker Compose. <https://docs.docker.com/compose/>

Koyeb. (s. f.). Documentation. <https://www.koyeb.com/docs>

Kubernetes. (s. f.). Service. <https://kubernetes.io/docs/concepts/services-networking/service/>

Railway. (s. f.). Deploy a Docker Compose app to production.

<https://docs.railway.com/guides/docker-compose>

Render. (s. f.). Docker on Render. <https://render.com/docs/docker>

Proyecto Heladería Monitoreo. (2026). Código fuente y documentación interna del repositorio.

Anexos y pendientes

Elemento	Estado
.dockerignore en cuatro servicios	Completado
Datos institucionales	[PENDIENTE]
URL pública	[PENDIENTE]
Capturas Docker/pruebas/despliegue	[PENDIENTE]
Video demostración	[PENDIENTE]
Resultados matriz de pruebas	[PENDIENTE]
Arquitectura AWS/IaC si se usa AWS	[PENDIENTE]